

Enroll:- 25CI2110042

Name:- Harsim Kharande

श्री स्वामिनारायण विद्यापीठ

Di. / / Pg. no. 1

Advance Java - Spring Boot

Theory Assignment - 1.

1) What is Spring Boot?

⇒ Spring Boot is a Java framework built on top of the Spring Framework that makes it easier and faster to create production-ready applications.

⇒ Spring Boot helps Java developers build applications quickly with minimal configuration, making development faster and easier than using the traditional Spring Framework alone.

⇒ It is heavily maintained by the Pivotal Team & enables developers to build microservices quickly with minimal setup code so they can focus on business logic.

2) List out features of Spring Boot

⇒ Spring Boot provides several features that make ^{every} application development simple, fast, and efficient.

i) Faster Development

⇒ Reduces boilerplate configurations. Developers write less setup code & focus on features.

ii) Loosely Coupled Dependencies

⇒ Spring Boot uses Dependency Injection, which reduces the dependency between classes.

⇒ This makes the application easier to maintain, test, and modify.

iii) Starter Packs

⇒ Spring Boot provides Starter dependencies, which are pre-configured sets of libraries for common functionalities.

E.g.:

- Spring-boot-starter-web
- Spring-boot-starter-data-jpa

iv) Auto Configuration

⇒ Spring Boot automatically configures the application based on the dependencies available in the project.

⇒ It minimizes manual configuration & setup.

v) Production-Ready Actuator

⇒ Spring Boot Actuator provides built-in features to monitor & manage applications.

vii) Embedded Server Support

⇒ Spring Boot comes with embedded web servers such as:

- Apache Tomcat
- Jetty
- Undertow

⇒ Application can run directly without installing an external server.

viii) Externalized Configuration

⇒ Configuration setting can be stored outside the application code.

- application.properties
- application.yml
- Environment Variables
- Command-line arguments

⇒ This makes it easy to use different configurations for development, testing, and production.

viii) Live Reload (Dev Tools)

⇒ Spring Boot Dev Tools automatically restarts the application when code changes are detected.

⇒ It supports Live Reload, allowing browser updates without manual refresh in supported setups, which speeds up development.

ix) Version Management

⇒ Spring Boot manages compatible versions of Spring libraries & third-party dependencies automatically.

⇒ Developers do not need to specify versions for most Spring dependencies, reducing version conflicts.

3) Which tools are required to build an application in Spring Boot?

⇒ To build a Spring Boot application, you need the following tools:

i) Java 8+

- Recommended version JDK 21 (LTS).

ii) Spring Boot

- The Framework used to develop Java-based web and enterprise applications.

- Provides auto-configuration, starter dependencies, and an embedded server.

iii) Build Tool

- A build tool manages project dependencies and builds the application

* Maven 3.6.3+ → Primary build tool.
Downloads dependencies from Maven

25CI2110042

2 Haxiom

શ્રી માધવિનાથ બાઈનરિયલ

Dt: / /

Pg no:

7

Central automatically.

* Gradle 6.6+ → Alternative build tool.
uses build.gradle instead of pom.xml.

iv) IDE.

- STS 4 / IntelliJ

- An IDE is used to write, edit, debug, and run Spring Boot applications.

- STS 4 recommended, IntelliJ IDEA Ultimate also fully supported.

4) List out the features of the 12-Factor app.

⇒ The 12-Factor APP is a methodology for building modern, Scalable, maintainable, and Cloud-native applications.

1) Codebase

- One Codebase tracked in Version Control.

- Multiple deployments use the same Codebase.

2) Dependencies

- Explicitly declare & isolate dependencies.
- A dependency is a Spring Boot application is maintained by pom.xml which comprises various dependencies which have their own version. The advantage of these dependencies is to include them as a separate component and not the whole library.

3) Config

- Store Config in the environment.
- All the configurations can be made environment-specific and can be called by running the Spring Boot application with the specific profile.

4) Backing Services

- Treat backing Services as attached resources.
- Your application may use the external resources like database, network firewalls and backing tools like redis.

5) Build, release, run:

- Strictly separate build and run stages.
- With defined pom.xml, you can upload your release build to the artifact repository.
- By simple build commands, you can build and deploy your applications.

6) Processes

- Execute the app as one or more Stateless Processes.

- Spring Boot allows your application to run independently.
- By Creating Restful APIs, you can develop the Services that don't require maintaining States of the process.

7) Port binding

- Export Services via Port binding.
- All web applications built using Spring Boot have the Capability to use the Embedded Tomcat Server by default exposing the Port 8080 So that the Services can be Consumed by other applications.

8) Concurrency

- Scale out via the Process model.
- The applications can also be Scaled up and Scaled down depending on the Users

hitting the environment.

9) Disposability

- Maximize robustness with fast startup and graceful shutdown.
- The Spring Boot application have the minimum startup time ranging from 8 secs to few minutes.
- when the application is integrated with the web server, that is, using embedded servers like Tomcat, it allows the application to be shut down gracefully so that any request that comes in during the shutdown process may be fulfilled successfully.

10) Dev/Prod Parity

- keep developing, staging, and production as similar as possible.

- It is essential to have the similar states of the environment on which the applications are running.
- This helps us to identify the errors coming to the lower environments & fix them before promoting the applications to the production environment.

11) Logs

- Treat logs as event streams.
- The application logs can be sent out as the stream to the log aggregator tools like Splunk, which indexes the logs based on the application.

12) Admin Processes.

- Run admin/management tasks as one-off processes.

- There can be few APIs exposed as REST end points, which can be only used by the users who have ADMIN roles.

5) What is the Parent dependency of a Spring Boot application? which annotations are included.

⇒ The Parent dependency of a Spring Boot application is Spring-boot-starter-parent.

<Parent>

<groupId>org.springframework.boot</groupId>

<artifactId>spring-boot-starter-parent

</artifactId>

<version>3.x.x</version>

</Parent>

- Manages dependency versions automatically.
- Provides default Maven Plugin configurations.
- Reduces manual configuration.
- Ensures compatibility between Spring Boot libraries.

માણસને પૂર્વગ્રહ સિવાય બીજો કોઈ ગ્રહ નહતો નથી.

The `@SpringBootApplication` annotation is a combination of the following three annotations:

1) `@SpringBootConfiguration`

- Indicates that the class is a Spring Boot Configuration class.
- It is a specialized version of `@Configuration`.

2) `@EnableAutoConfiguration`

- Enables Spring Boot's auto-configuration feature.
- Automatically configures beans based on the dependencies present in the project.

3) `@ComponentScan`

- Scans the current package & its sub-packages for Spring components such as:
 - `@Controller`
 - `@Service`

25CI2110042

Haxiom

श्री स्वामिनारायण विवाहन मिशन

DL 211 Pg. no. 15

- @Repository
- @Component

7) What is transitive dependency and how can you capture all dependencies in a project?

⇒ A transitive dependency is a dependency that is automatically included because it is required by another dependency in your project. you do not add it directly to your pom.xml; Maven downloads it automatically.

Example.

<dependency>

<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-web</artifactId>

</dependency>

When you add Spring-boot-starter-web, Maven automatically downloads other required libraries such as:

- Spring Core, - Spring MVC, Embedded Tomcat,
- Jackson, - Spring Boot Starter.

⇒ These automatically downloaded libraries are called transitive dependencies.

You can capture or view :

1) Maven Dependency Tree

⇒ mvn dependency:tree

- Displays all direct and transitive dependencies in a tree structure.

2) Maven Dependency List

⇒ mvn dependency:list

- Gets all dependencies used in the project.

3) IDE Support

⇒ IDEs such as Spring Tool Suite, IntelliJ IDEA, and Eclipse can display the project's dependency hierarchy through the Maven project view.

2] How can you change the name of the bean?

⇒ In Spring boot, the bean name is the identifier used by the Spring Container to manage a bean.

1] Using `@Component` with a custom name.

⇒ Specify the bean name as the annotation value.

```
import org.springframework.stereotype.Component;
@Component("StudentBean")
```

```
public class Student { }
```

2] Using the `@Bean` Annotation

⇒ When defining a bean in a Configuration class, specify the bean name in the name attribute.

⇒ The bean will be registered with the name StudentBean.

3) Default Bean Name

⇒ If no name is specified, Spring uses the class name with the first letter in lowercase.

@Component

public class StudentService

{ }

2) How would you know if any keyword is used in an annotation?

⇒ In Spring Boot, annotations use the @Symbol.

⇒ If you see a word preceded by "@", it indicates that it is an annotation.

Example.

@Component

@Service

@Repository

@Controller

- Annotation names are always prefixed with @.
- They provide metadata to the compiler or the Spring framework.

10) How do you identify which annotation can be used at the top of class, fields?

⇒ you can identify where an annotation can be used by checking its @Target meta-annotation.

⇒ The @Target annotation specifies the type of Java element on which an annotation is allowed.

```
@Target(ElementType.TYPE)
public @interface Component {
}
```

⇒ This means @Component can only be placed on a class.

@Target (ElementType.FIELD)
Public @interface Autowired { }

⇒ This means @Autowired can be placed on a field.

11) Which are the Stereotype annotations?

⇒ Stereotype Annotations are special annotations in Spring that identify the role of a class and allow Spring to automatically detect & register it as a bean in the application context.

1) @Component.

- A general-purpose Stereotype annotation.
- Used to mark a class as a Spring

managed Component.

2) @Service.

- Used for the Service layer of an application.
- Contains business logic.

3) @Repository

- Used for the data access layer.
- Handles database operations.
- Also provides automatic exception translation for database errors.

4) @Controller

- Used in Spring MVC applications.
- Handles HTTP requests & returns Views.

⑤ @RestController.

- Used to Create REST APIs that return data.

12) How do you Schedule a Spring job?

⇒ In Spring Boot, a job can be scheduled using the @Scheduled annotation, provided by Spring.

⇒ It allows a method to execute automatically at a fixed time, fixed interval, or according to a cron expression.

1) Enable Scheduling.

⇒ Add @EnableScheduling in the main Spring Boot application class.

Harrison

2) Create a Scheduled Method.

⇒ Use the `@Scheduled` annotation on the method that needs to run automatically.

13) Does the Spring job Support the return type?

⇒ No, Spring Scheduled jobs do not support return types.

⇒ The method annotated with `@Scheduled` must have a void return type.

⇒ Because Spring executes the method automatically and does not handle or store the returned value.

- The scheduled method should not accept parameters.
- It should normally be public.

@Component

public class ScheduledTask

@Scheduled(fixedRate = 5000)

public void executeTask()

{

 System.out.println("Scheduled task");

}

}

14] How does Spring know where to look for beans?

⇒ Spring knows where to look for beans through Component Scanning.

⇒ It scans specific packages to find classes annotated with Spring

Stereotype annotations & registers them as beans in the Spring Container.

1) Using @ComponentScan

- The @ComponentScan Annotation tells Spring which packages to scan.
- Spring will scan the com-example.service package and register eligible classes as beans.

2) using @SpringBootApplication

⇒ In Spring Boot, the main class:

@SpringBootApplication

public class Application

{

public static void main(String[] args)

{ SpringApplication.run(Application.class, args);

}

}

3) Using Java Configuration (@Bean)

⇒ Beans Can also be manually defined using @Bean.

```
@Configuration
public class AppConfig
{
```

```
    @Bean
```

```
    public Student Student()
    {
```

```
        return new Student();
    }
```

```
}
```

15] How Can you Save data in the database?

⇒ In Spring Boot, data can be saved into a database using Spring Data JPA and a repository interface.

25.12.110042

Haxiom

27

1) Create an Entity class

- Use @Entity to map a Java ~~code~~ Class with a database table.

@Entity

Public class Student

{

 @Id

 Private int id;

 Private String name;

}

2) Create a Repository interface

- Extend JpaRepository to get built-in database operations.

@Repository

Public interface StudentRepository extends

JpaRepository<Student, Integer>

{

}

Spring Repository provides method:

- Save()
- findById()
- findAll()
- delete()

3) Use the Save() Method.

⇒ Inject the repository into a Service or Controller & Call the Save() method.

@Service
Public class StudentService {

@Autowired
Private StudentRepository repository;

Public Student SaveStudent (Student
Student)

{
return repository.save (Student);

}

25CI2110042

Harioom

શ્રી સ્વામિનરાયણ દિવાન મંદિર

DLN / 14 / 2019 / Pg no.

29

41 Save Data Through Controller.

@ Rest Controller

Public class StudentController

{

@Autowired

Private StudentService Service;

@PostMapping("/students")

Public Student addStudent

(@RequestBody Student student)

{

return Service.saveStudent
(student);

}

}

16) How Can you Select all records from Spring data JPA?

⇒ In Spring Data JPA, you can select all records from a database table using the built in `findAll()` method provided by the `JpaRepository` interface.

Step 1: Create Repository Interface

```
@Repository
public interface StudentRepository
    extends JpaRepository<Student,
        Integer>
    {
        3
    }
```

Step 2: use `findAll()` Method.

```
@Service
public class StudentService
    {
        2
    }
```


25012110042

Harionm

શ્રી સ્વામિનારાયણ ઉચ્ચશિક્ષણ મિશન

DL 1/11/21 Page: 31

@ Authorized
Private Student Repository repository;
Public List < student > get AllStudents
& return repository.findAll();

Step 3: Access Through Controller

@ Rest Controller
Public class StudentController

@ Authorized
Private StudentService service;

@GetMapping("/students")
Public List < Student > getStudents

& return service.getAllStudents();

17) How Can you Create Custom query methods for filtering records?

⇒ In Spring Data JPA, Custom query methods can be created for filtering records by defining methods in the Repository interface.

⇒ Spring Data JPA, Custom automatically generates queries based on the method name.

Example

@Entity

Public Class Student

{

@Id

Private int id;

Private String name;

Private String email;

Private String city;

}

18) How Can you auto-generate the value of a primary key using persistence API?

⇒ In JPA, you can auto-generate the value of a primary key using the `@GeneratedValue` annotation along with the `@Id` annotation.

Example.

`@Entity`

`Public class Student`

`@Id`

`@GeneratedValue(strategy = GenerationType.IDENTITY)`

`Private int id;`

`Private String name;`

`Private String email;`

3

- @ID : Marks the field as the Primary key.
- @GeneratedValue - Automatically generates the primary key value.
- GenerationType.IDENTITY: uses the database's auto-increment feature to generate the ID.

19) How do you mark an attribute primary key for the table?

⇒ In JPA, you mark an attribute as the primary key by using the @Id annotation.

Example.

```
import jakarta.persistence.Entity;
import jakarta.persistence.Id;
```


@Entity

Public class Student {

{

@Id

private int id;

private String name;

private String email;

}

- @Entity - Marks the Class as a database entity.
- @Id Marks the id field as the Primary key of the table.
- Each entity must have one Primary key.
- If the Primary key should be auto-generated.
- You can combine @Id with @GeneratedValue.

20/ What are different types of HTTP methods?

⇒ HTTP methods define the action that a Client wants to perform on a Server or resource.

1) GET

- Used to retrieve data from the Server.
- Does not modify data.

E.g.

GET /students

2) POST

- Used to Create or insert new data on the Server.

25C12110042

Hariom

શ્રી સ્વામિનારાયણ ડિવાઇન મિશન

DL. 11/7

Pg.no. 37

E.g.:

POST /students

3) PUT

- used to update an existing resource.
- Replaces the entire resource with the new data
- E.g.:-

PUT /students/1

4) DELETE

- used to delete a resource from the server.

E.g.:-

DELETE /students/1

5) PATCH

- Used to Partially update an existing resource.
- Updates only the Specified field instead of replacing the entire resource.
- E.g.

PATCH /students /1

6) TRACE

- used for diagnostic purposes.
- Returns the received request so the client can see what internal state changed.

21] How to Create GET RESTful API?

⇒ A GET RESTful API is used to retrieve data from the Server.

⇒ In Spring boot, it is created using the @GetMapping annotation.

Step 1:- Create an Entity

@Entity

Public class Student {

@Id

Private int id;

Private String name;

}

Step 2:- Create a Repository

@Repository

Public interface StudentRepository

extends JpaRepository<Student, Integer>

{ }

માણસને પૂર્વગ્રહ સિવાય બીજો કોઈ ગ્રહ નહતો નથી.

25CI2110042
Harion

શ્રી સ્વામિનારાયણ ટ્રસ્ટ મિશન

DL

Pg.no. 40

Step 3:- Create a Service

@Service

Public Class StudentService

{

@Autowired

Private StudentRepository
repository;

Public List<Student> getAllStudents()

{

return repository.findAll();

}

}

22)

=>

Step 4:- Create a Controller

@RestController

@RequestMapping("/students")

Public Class StudentController

{

// Code -

}

Step 5: Test the API

- Run the Spring Boot application and send a GET request:
- GET `http://localhost:8080/students`

22) How to Consume POST API?

⇒ A Post API is Consumed by Sending data from the Client to the Server. The data is usually sent in JSON format in the request body.

⇒ In Spring boot, the Server receives this data using the `@RequestBody` annotation.

⇒ Example Post API (Server Side)

```
@RestController
@RequestMapping("/students")
public class StudentController {
```

@ Post Mapping
 Public String addStudent (@ Request
 Body Student Student)

return "Student added successfully"

Consume the POST API Using
 Postman

- 1) open Postman
- 2) Select the POST method.
- 3) Enter the URL.
- 4) Go to Body → raw → JSON.
- 5) Enter the JSON Data.
- 6) Click Send.

Response:-

- Student added successfully.

331 How to Create the Custom health Check for the application?

⇒ In Spring Boot, you can create a Custom health Check using Spring Boot Actuator by implementing the HealthIndicator interface.

1) Add Actuator Dependency.

2) Create a Custom Health Indicator.

@Component
Public class CustomHealth implements HealthIndicator

{

@Override

Public Health health()

{

//Statement

- build();

}

}

3) Check the Health End Point.

=> Run the application & open:

`http://localhost:8080/actuator/health`

24) How to handle dynamic query parameters?

=> In Spring Boot, dynamic query parameters are handled using the `@RequestParam` annotation.

Example.

```
@RestController
public class StudentController {
```

```
    @GetMapping("/students")
```

```
        // Statements
```

```
    }
```

```
}
```


⇒ API Request.

⇒ GET `http://localhost:8080/students` ?

25) What is the other annotation used in conjunction with?

⇒ `@RequestParam` is used to read query parameters from the URL.

- `@PathVariable` is used to read values from the URL path.

Example.

`@GetMapping("/students/{id}")`

`public Student getStudent(@PathVariable int id)`
`return Secure;`

`getStudent(id);`

3

26) How to enable basic Security in a Spring boot application?

⇒ Spring boot provides Spring Security to secure applications.

⇒ Basic Security can be enabled by adding the Spring Security dependency and configuring user credentials.

Step 1:- Add Spring Securing Dependency

Step 2:-

⇒ Configure username and Password.

⇒ Add in application.properties.

Step 3:-

After adding the dependency, Spring Boot automatically enables Basic Authentication.

⇒ Spring Security displays a login page or requires HTTP Basic authentication.

24] Who decides the authorization roles for a trusted client?

⇒ The Authorization Server decides the authorization roles & permissions for a trusted client in OAuth 2.0.

How it works:

- The Client requests access to a protected resource.
- The Authorization Server authenticates the user.
- It verifies what permissions the client is allowed to have.

- It then issues an access token containing those permissions.
- The Resource Server checks the token and grants or denies access accordingly.

28) What is OAuth 2.0 Security and how does it work?

⇒ OAuth 2.0 is an authorization framework that allows a third-party application to access a user's protected resources without sharing the user's password.

How OAuth 2.0 works

- User requests to access a protected resource.
- The Client Application redirects the user to the Authorization Server.

- The user logs in and grants permission.
- The Authorization Server issues an Access Token.
- The Resource Server validates the token and returns the requested data.

29/ What are the two parts of OAuth 2.0 implementation?

⇒ 1) Authorization Server.

- Authenticates the user.
- Verifies the client's identity
- Issues Access Tokens.
- Decides the user's permissions.

2) Resource Server.

- Hosts the protected resources
- Validates the access token received from the Client.
- Grants or denies access based on the token.

3) How to disable Security in Spring Boot?

⇒ You can disable Spring Security by configuring a Security filter chain that permits all requests.

Example: -

```
@Configuration
public class SecurityConfig
```

```
@Bean
public SecurityFilterChain (HttpSecurity http)
```


25CT 2100 42

Harion

શ્રી સ્વામિનારાયણ ઉવાચન મિશન

Pg no 53

throws Exception

http

(get of access disabled)

;;

return http build();

3

2

3) What are multipart requests?

=> Multipart requests are HTTP requests used to send multiple types of data in a single request, such as files.

- File uploads
- Sending form fields together with files.

Harrison

Handling Multipart Requests In Spring Boot.

- Use the `@RequestParam` annotation with the Multipart File class.

Ques MCQ.

1) Which Spring annotation is used to create RESTful web services using Spring MVC?

⇒ `@RestController`.

2) `@RestController` annotation is a combination of the below two annotations.

⇒ `@Component`
`@ResponseBody`
annotations.

3). Which Spring annotation is used to handle HTTP POST requests?

⇒ @PostMapping

4). Which Spring annotation is used to handle HTTP GET requests?

⇒ @GetMapping

5). Which is the default HTML template in Spring Boot.

⇒ Thymeleaf

6). Which Starter dependency is used to develop web applications or RESTful web services.

⇒ Spring-boot-starter-web

75. How can you specify the port on which a Spring Boot application runs?

⇒ All of the above.

8). What is the @SpringBootApplication annotation used for?

⇒ To enable Spring Boot auto-configuration.

9). Minimum Java version used for Spring boot 3?

⇒ Java 17.

— X — X — X — X —
— Completed —